

TCS - 504 SYSTEM DESIGN - ASSIGNMENT 1

A. Requirement Analysis

Functional Requirements (FR)

- FR1 : The system shall list all currently playing movies with their basic details (e.g. title, language, genre).
- FR2 : When a movie is chosen, the system shall show all available shows for that movie with screen number and start time.
- FR3 : When a show is chosen, the system shall display the seat layout of the screen with status of each seat as AVAILABLE or BOOKED.
- FR4 : The system shall allow the customer to book one or more free seats. It shall not allow booking of already booked seats and shall show an error message.
- FR5 : The system shall support different seat categories with fixed prices : SILVER - ₹150, GOLD - ₹250, PLATINUM - ₹400.
- FR6 : The system shall accept payments via UPI, Card or Cash. If payment fails, the booking must not be confirmed and the seats must remain available.
- FR7 : After a successful booking, the system shall generate a ticket with booking id, movie name, screen, show time, seat numbers and total amount.
- FR8 : The system shall allow cancellation of a booking. On cancellation, the seats shall be marked as available again.

Non-Functional Requirements (NFR)

- NFR1 : Modularity - The system shall be modular with well defined classes and responsibilities.
- NFR2 : Extensibility - The system shall be extensible to add new features (e.g. new payment method).
- NFR3 : Input validation and robustness - The system shall validate all inputs and handle invalid inputs gracefully.
- NFR4 : Maintainability - The system shall be easy to understand, test and maintain.

C. Classes - data, methods, visibility, knows/does/must NOT do

1. Movie

- id : int
- title : String
- language : String
- genre : String
- duration : int
- + getDetails() : String
- + getId() : int
- + getTitle() : String

Knows : its own details.

Does : provides movie information.

Must NOT do : manage shows or bookings.

2. Seat

- seatNo : String
- category : String(SILVER/GOLD/PLATINUM)
- price : double
- + getSeatNo() : String
- + getCategory() : String
- + getPrice() : double

Knows : own seat number and category.

Does : provides seat information.

Must NOT do : know about bookings or customers.

3. Screen

- screenNo : int
- seats : List<Seat>
- + getScreenNo() : int
- + getSeats() : List<Seat>
- + getSeat(String seatNo) : Seat

Knows : its seats.

Does : manages seats.

Must NOT do : handle bookings or payments.

4. Cinema

- id : int
- name : String
- location : String
- screens : List<Screen>
- + getDetails() : String
- + getScreens() : List<Screen>
- + addScreen(Screen s) : void

Knows : its screens.

Does : manages screens.

Must NOT do : handle bookings directly.

5. Show

- id : int
- movie : Movie
- screen : Screen
- startTime : LocalDateTime
- showSeats : List<ShowSeat>
- + getDetails() : String
- + getShowSeats() : List<ShowSeat>
- + getAvailableSeats() : List<ShowSeat>

Knows : movie, screen and seats.

Does : provides show information.

Must NOT do : process payments or print tickets.

6. ShowSeat

- seat : Seat
- status : String (AVAILABLE/BOOKED)
- booking : Booking (nullable)
- + isAvailable() : boolean
- + book(Booking b) : void
- + cancel() : void
- + getSeat() : Seat

Knows : seat and its status.

Does : marks seat as booked/available.

Must NOT do : handle payments.

7. Customer

- id : int
- name : String
- phone : String
- email : String
- + getDetails() : String
- + getId() : int

Knows : its own details.

Does : provides customer information.

Must NOT do : process bookings directly.

8. Booking

- bookingId : long (static)
- customer : Customer
- showSeats : List<ShowSeat>
- totalAmount : double
- + status : String (CONFIRMED/CANCELLED)
- + getBookingId() : long
- + getShowSeats() : List<ShowSeat>
- + getTotalAmount() : double
- + cancel() : void

Knows : customer, seats and amount.

Does : maintains booking details.

Must NOT do : print tickets or process payments.

Note :

- Encapsulation : keep attributes private and provide public methods.
- Use static variable for generating unique bookingId.
- Use this keyword to refer to current object, e.g. this.bookingId.

9. Payment <<abstract>>

- amount : double

+ pay(double amount) : boolean (pure virtual)

+ getAmount() : double

Knows : amount.

Does : defines common payment behaviour.

Must NOT do : know specific payment details (implemented in subclasses).

10. UpiPayment (extends Payment)

- upiId : String

+ pay(double amount) : boolean

Knows : UPI id.

Does : processes UPI payment.

Must NOT do : handle other payment types.

11. CardPayment (extends Payment)

- cardNumber : String

+ pay(double amount) : boolean

Knows : N/A.

Does : accepts cash payment.

Must NOT do : handle other payment types.

12. CashPayment (extends Payment)

+ pay(double amount) : boolean

Knows : N/A.

Does : accepts cash payment.

Must NOT do : handle other payment types.

13. PriceCalculator

+ calculatePrice(List<ShowSeat> seats) : double

Knows : seat categories and rules.

Does : calculates total price.

Must NOT do : process payments.

14. TicketPrinter

+ printTicket(Booking b) : void

Knows : booking details.

Does : generates and prints ticket.

Must NOT do : process payments
or modify bookings.

15. BookingService

- priceCalculator : PriceCalculator

+ TicketPrinter : TicketPrinter

+ listMovies() : List<Movie>

+ listShows(int movieId) : List<Show>

+ showSeats(int showId) : List<ShowSeat>

+ bookSeats(int showId, List<String> seatNos, Customer c) : Booking

+ cancelBooking(long bookingId) : void

Knows : shows, seats, customers and payments.

Does : coordinates the complete booking flow.

Must NOT do : implement payment details (uses Payment abstraction).

16. CinemaSystem / Main Menu (UI)

- bookingService : BookingService

+ start() : void

+ showMenu() : void

Knows : bookingService.

Does : handles user input and menu navigation.

Must NOT do : contain business logic (kept in service classes).

OOP Concepts used :

- Encapsulation : private attributes, public methods.
- Abstraction : Payment is an abstract class.
- Inheritance : UpiPayment, CardPayment, CashPayment extend Payment.
- Runtime polymorphism : Payment p = new UpiPayment(); p.pay().
- Compile-time polymorphism : e.g. overloaded methods (possible).
- Static variable : static long bookingId in Booking.
- this keyword : this.attribute used in constructors/methods.
- Composition : Cinema → Screen, Screen → Seat, Show → ShowSeat.
- Aggregation : Show → Movie, Booking → ShowSeat.
- Association : Show → Screen, BookingService → Booking.

B. Noun - Verb Analysis

Noun found	Keep as a class?	Reason
Movie	Yes	Has data and behaviour
Seat	Yes	Represents a seat with status and type
Screen	Yes	Collection of seats
Cinema	Yes	Owns screens
Show	Yes	Links movie, screen and time
ShowSeat	Yes	Represents a seat in a particular show with status
Customer	Yes	Customer details
Booking	Yes	Stores booking information
Payment	Yes (abstract)	Abstract concept for all payment types
UpiPayment	Yes	Concrete payment type
CardPayment	Yes	Concrete payment type
CashPayment	Yes	Concrete payment type
PriceCalculator	Yes	Calculates price based on seat category
TicketPrinter	Yes	Generates and prints ticket
BookingService	Yes	Coordinates the booking flow
seat layout	No	It is a view of Show's seats, not a domain entity
menu	No	UI/input flow, not a domain entity
ticket	No	Formatted output, not an owned entity

Verbs from requirements become methods in relevant classes, e.g.:
 listMovies(), listShows(), showSeats(), bookSeat(),
 pay(), cancelBooking(), printTicket().